# Assignment 1

PCCST501 – Computer Networks

"How the Internet Delivers a WebPage"

A Case Study on YouTube (www.youtube.com)

**Name**: Naveen Mohanan

**Class:**S5 CSA

**Roll No:**50

**Submitted To:** Arsha Harikrishnan

## 1. The TCP/IP Protocol Stack and the Role of Each Layer

Every exchange of data on the Internet, including something as ordinary as opening YouTube, is organised through a layered model known as the TCP/IP suite. It splits the overall task of communication into five independent layers, where each layer relies on the services offered by the layer directly beneath it and, in turn, offers its own service upward.

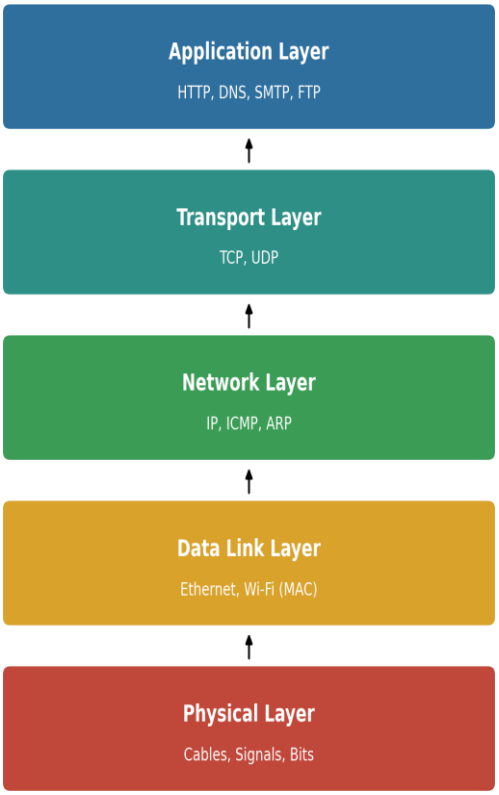



*Fig. 1: The five-layer TCP/IP stack involved in loading the YouTube homepage*

- **Application Layer** – This is the layer closest to the end user. It hosts the protocols an application actually talks in – HTTP for fetching the page, DNS for resolving the domain name, along with others such as SMTP and FTP.
- **Transport Layer** – This layer is responsible for moving data between the correct processes on the source and destination machines. TCP, used here, breaks the data into segments and guarantees ordered, dependable delivery between the browser and YouTube's server.
- **Network Layer** – Using the Internet Protocol, this layer attaches logical (IP) addresses to each packet and determines the path it should take across the chain of routers separating the user from YouTube's infrastructure.
- **Data Link Layer** – Operating over a single physical link such as a Wi-Fi or Ethernet connection, this layer packages data into frames and tags them with MAC addresses so that neighbouring devices can exchange them correctly.
- **Physical Layer** – The lowest layer converts frames into a stream of raw bits and transmits them physically as electrical pulses, light signals, or radio waves through the transmission medium.

## 2. The Role of DNS in Resolving a Domain Name to an IP Address

Machines on the Internet identify one another using numeric IP addresses rather than names, so typing www.youtube.com into a browser cannot, by itself, establish a connection. The Domain Name

System (DNS) exists to bridge this gap: it is an application-layer service that behaves like a distributed directory, translating memorable domain names into the IP addresses that routing actually depends on.

1. The browser first checks its own cache, and that of the operating system, to see whether the IP address for the domain has already been resolved recently.
2. If no cached entry exists, the query is forwarded to a DNS resolver, typically operated by the user's Internet Service Provider.
3. The resolver, in turn, queries a root server, then a .com top-level-domain server, and finally YouTube's authoritative name server, to obtain the correct address.
4. The resulting IP address is passed back to the browser and stored temporarily (governed by a Time-To-Live value) so that repeated visits do not need to repeat the entire lookup.

Once this lookup completes, the browser has the actual network location of YouTube's servers and can begin establishing a connection to them.

## 3. How HTTP Retrieves the Requested Webpage

HTTP (HyperText Transfer Protocol) governs how the browser and YouTube's server exchange the actual content of the page, following a request-response pattern built on top of an established TCP connection over port 80, or port 443 when the connection is secured with HTTPS.

- **Request:** The browser issues a GET request that includes a request line (identifying the method, URL, and HTTP version) together with header fields such as Host and Accept-Language.
- **Response:** The server answers with a status line (for example, 200 OK), a set of header fields describing the content, and a message body carrying the HTML, CSS, JavaScript, or media being requested.
- **Multiple exchanges:** Because a page like YouTube's homepage embeds many separate resources – images, scripts, stylesheets – the browser issues several GET requests, generally reusing a single persistent HTTP/1.1 connection rather than opening a fresh one each time.

## 4. How TCP Underpins Reliable Delivery for HTTP

HTTP is deliberately built on top of TCP because TCP is connection-oriented and guarantees reliable delivery. Before any HTTP data is exchanged, a three-way handshake (SYN, SYN-ACK, ACK) sets up the connection between browser and server. TCP then contributes several guarantees that HTTP itself does not provide:

- **Error control:** Sequence numbers and acknowledgements let the receiver detect segments that were lost or corrupted in transit, triggering retransmission where necessary.
- **Ordered delivery:** The same sequence numbers allow the receiving side to reconstruct the segments in their original order, even if they arrive out of sequence.
- **Flow control:** The rate at which the sender transmits data is matched to the available buffer space at the receiver, preventing the receiver from being overwhelmed.
- **Congestion control:** TCP adapts its sending rate in response to the current level of congestion on the network path.

## 5. Client–Server versus Peer-to-Peer Architecture

YouTube is built on the Client–Server model rather than a peer-to-peer design. Its video catalogue, metadata, and comments are hosted centrally on servers and content-delivery networks operated by Google, which remain continuously available. The browser acts purely as a client: it requests content,

receives it, and has no role in serving that content to anyone else. Unlike peer-to-peer systems such as BitTorrent, where every participant can both consume and redistribute data, the roles of client and server on YouTube are fixed and never reverse.

## 6. Comparing HTTP and FTP

| Basis | HTTP | FTP |
|---|---|---|
| **Purpose** | Requests and delivers web pages and embedded resources | Transfers files between a client and a server |
| **Connections** | Ordinarily a single, persistent TCP connection (HTTP/1.1) | Two separate connections – one for control (port 21), one for data (port 20) |
| **State** | Stateless by design; continuity across requests is simulated with cookies | Stateful; the server tracks the session and current directory |
| **Port** | 80 for HTTP, 443 for HTTPS | 21 for control, 20 for data |
| **Typical use** | Optimised for small, frequent request/response exchanges | Optimised for transferring large files in ASCII or binary mode |

## 7. How Cookies Improve the User Experience

HTTP itself has no memory of previous interactions between a particular browser and a server. Cookies exist to compensate for this: a server sends a Set-Cookie header once, and from then on the browser automatically attaches that cookie to every later request made to the same domain, allowing the server to recognise the client without asking for the same information again.

**Example on YouTube:** after a user signs in and watches a handful of videos, a cookie preserves their login state and viewing preferences. The next time they visit, YouTube uses that cookie to keep them signed in, apply their preferred language and region, resume recommendations based on prior watch history, and retain autoplay and video-quality settings, all without the user having to re-enter this information.

## 8. End-to-End Communication Flow

Bringing the previous sections together, the diagram below traces the complete path taken from the moment a user types a URL to the moment the page appears on screen: DNS resolves the domain name, TCP establishes a reliable connection, HTTP carries the request and the response, and the browser finally renders the received content.

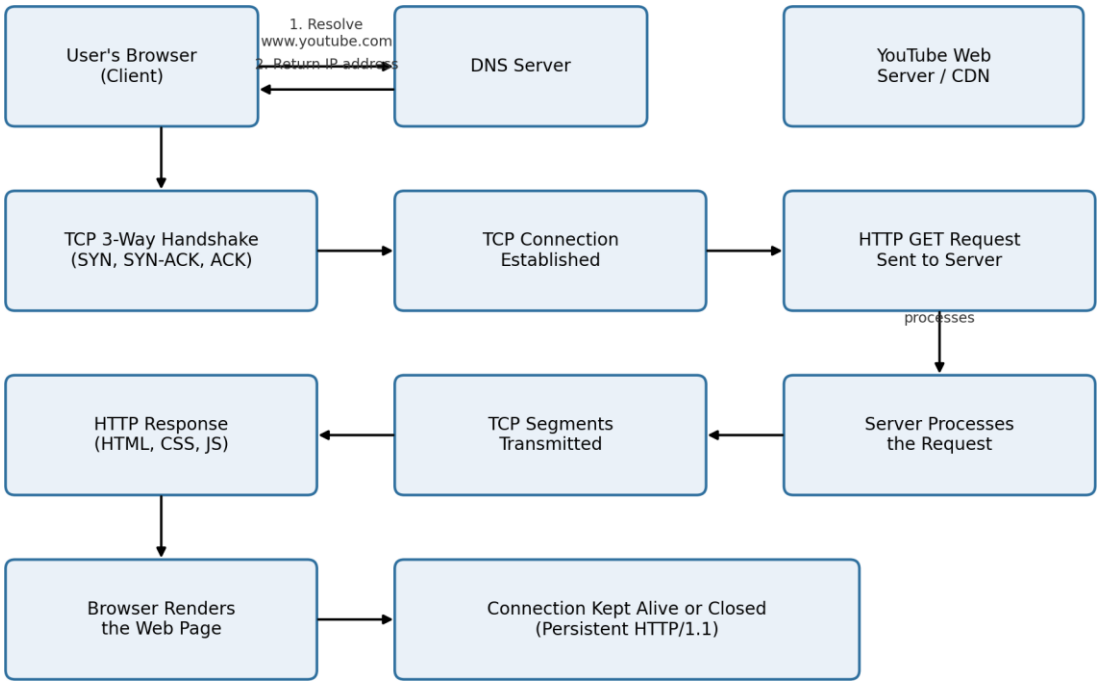



*Fig. 2: End-to-end communication flow between the browser and YouTube's server*

## 9. Conclusion

Loading a single YouTube page depends on several application-layer protocols working in close coordination. DNS removes the burden of remembering numeric addresses by mapping domain names to IP addresses, while HTTP defines the language in which the browser and server exchange requests and the corresponding web content. FTP, although not used in this particular case study, plays a comparable role whenever files need to move directly between systems, and cookies add a sense of continuity to otherwise stateless HTTP exchanges by letting servers remember individual users. All of this activity is carried on top of the layered TCP/IP architecture, with TCP in particular ensuring that data arrives completely, in order, and at a pace the receiver can handle. Taken together, these protocols allow an enormous amount of technical complexity to remain invisible, delivering the smooth, near-instant browsing experience that billions of Internet users depend on every day.